

CTO Audit: What's Actually Wrong With The PRD, And The Rebuild

Read this before you write a single line of code. Everything below assumes you have ~48 hours, a small team, one laptop each, and venue wifi you should not trust.

PART 1 — THE AUDIT

The original PRD is a good *design*. As a 48-hour *build plan* it has a specific, recurring failure pattern: every "real government data source" I cited as available is available to *read* on a website, not to *pull programmatically at scale on demand* — and the difference between those two things is where hackathon teams lose a full day they don't have. Here is every load-bearing assumption that doesn't survive contact with 48 hours, in order of how badly it would hurt you.

#	Original element	Why it fails in 48h	Verdict
1	FR-1: RF/XGBoost trained on the NRSC landslide inventory	The inventory lives inside Bhuvan's web GIS. Bulk/API export is by request to NRSC — that's a government email thread with a response time measured in weeks, not a <code>pip install</code> . You will not have this data by hour 4, and without it there is no training set.	Cut the dependency. Model becomes stretch-goal, not critical path (see Part 2).
2	Coastal erosion module	Real shoreline-change analysis (DSAS-style) needs multi-date imagery, a trained analyst, and is a multi-week task even with data in hand. Nobody builds this credibly in a hackathon.	Cut entirely. Pick a non-coastal demo district so the gap never shows.
3	CartoDEM via Bhuvan/Bhoonidhi	Government geoportal logins are slow, sometimes down, and account approval can take time you don't have. This is a classic "burned 3 hours on a login screen" hackathon failure.	Cut. Use SRTM 30m (OpenTopography/USGS EarthExplorer) or Copernicus GLO-30 — no India-specific account needed, instant download.
4	IMD real-time API / Bhoonidhi API	Both explicitly require registration/whitelisting. There is no scenario where this is approved inside a hackathon window.	Cut from runtime entirely. Use only the free historical gridded rainfall archive (<code>IMDLIB</code>) for one real, named extreme-rainfall date, and <i>replay</i> it live. You were already half-doing this — now make it the only path, not a fallback.
5	PostGIS + Docker Compose	A DB server is a live dependency at demo time — if it doesn't start (dead battery, updated Docker, no internet to pull an image at the venue), your demo doesn't start. For ~30–50 villages and ~5 sites this is also just the wrong tool: you don't need a spatial database, you need a spreadsheet with lat/lon columns.	Cut PostGIS. In-memory GeoPandas/pandas over flat CSV/GeoJSON files. Zero servers, zero network dependency, runs fully offline on one laptop.
6	Raster tile server (titiler) for hazard layers	Standing up tile serving is real infrastructure work that produces zero extra judge-visible insight versus a simpler alternative.	Cut. Render each habitation as a colour/size-coded point on the map. Same information, an hour of work instead of a day.

#	Original element	Why it fails in 48h	Verdict
7	Self-hosted OSRM routing	Building a road-network graph + contraction hierarchy is a multi-hour infra task with a real chance of just not working by demo time.	Cut. Straight-line (haversine) distance, labelled honestly on-screen as "distance shown as the crow flies — turn-by-turn routing is a planned v2 feature." Judges respect an honest label far more than a broken routing widget.
8	Full RBAC + persisted audit-log table + 3 mocked logins	This is backend surface area that adds zero visible technical impressiveness — judges are scoring the engines, not your login screen.	Cut real auth. A single "View as: State / District / Analyst" dropdown that filters the UI. Keep the <i>human-approval click</i> (that's a real, important story beat) but log it to an in-memory list, not a database table.
9	React + MapLibre custom frontend, talking to a separate FastAPI backend	Two repos, a build step, CORS, a deploy step — meaningfully more coordination risk for a small team, for a stack the team has <i>less</i> proven experience shipping fast in than the alternative below.	Flip the recommendation (see Part 2): Streamlit + PyDeck/Folium as the single-process demo app.
10	GDAL/Rasterio running live inside the demo app	GDAL installs are a well-known hackathon time-sink, especially on Windows — wheel mismatches, missing system binaries, an hour gone before you've written a line of product code.	Do all raster work once, ahead of the live app , in a throwaway notebook (Colab is fine). Ship only the <i>output</i> — a CSV of per-village slope/HAND/rainfall values — into the demo. The live app never imports GDAL.
11	Real village polygon boundaries	Complete, clean village-polygon shapefiles matched to Census codes are genuinely hard to source quickly — community projects like <code>datameet</code> don't cover every state, and what they do cover isn't always clean.	Cut polygons. Use point centroids only (lat/lon), sized by population on the map. Trivial to build, visually just as clear.
12	Automated bulk Census Village Directory ingestion	Real Census bulk exports are messy — inconsistent codes across release years, awkward Excel formats. Writing a robust parser for this is its own multi-hour project.	Hand-curate. A 20–30 row CSV for your chosen district — population, households, distance to nearest health facility/road — sourced from the published District Census Handbook, OpenStreetMap, and Google Maps distances.

#	Original element	Why it fails in 48h	Verdict
			Slower to gather per-row, faster overall, and you control data quality.
13	SHAP explainability	Extra dependency, extra compute, and scikit-learn's built-in <code>feature_importances_</code> already answers "why" for a judge — SHAP buys you almost nothing extra for the demo.	Cut.
14	Cloud VM deployment as the primary demo path	Venue wifi failing mid-demo is one of the single most common ways hackathon teams lose points on presentation day, for reasons that have nothing to do with their engineering.	Demo runs entirely on the presenter's laptop, offline-capable. A pre-tested cloud link can exist as a backup, never as the primary.
15	Optimization engine (MIP village→site allocation)	No external data dependency — it's math over your own hand-curated inputs. Nothing here breaks.	Keep, untouched. This is your centerpiece.
16	Priority-score formula, explainability-by-construction	Pure arithmetic, zero infrastructure risk.	Keep, untouched.
17	Live "inject a rainfall event" recompute demo	Recomputes over data you already control locally — no live dependency at all.	Keep — this is now your best "wow" moment, precisely because everything riskier around it just got cut.

The pattern, stated once so it doesn't repeat: every item cut above was cut because it depended on *someone else's server, approval process, or infrastructure responding correctly on a specific afternoon*. Every item kept is cut from a different cloth — it's your own math over your own data. That's not a coincidence; it's the actual rule for what's safe to build in 48 hours.

On AI usage specifically — don't over-correct into "no ML at all," or you hand judges rejection risk #1 for free. The honest middle path: manually eyeball Bhuvan's public (browsable, not bulk-API) landslide map for your chosen district and note down ~15–20 landslide coordinates by hand (an hour of work, not a data pipeline), generate a similar number of non-landslide points well outside those buffers, pull slope/aspect/rainfall/NDVI at each from your pre-computed CSV, and train a small Random Forest on that. It's honestly a small-N proxy model — say so on stage, in exactly those words, as

evidence of technical maturity rather than something to hide. That is a real, defensible, achievable use of ML, as distinct from claiming you trained on "NRSC's inventory" when you actually couldn't get it.

PART 2 — THE REBUILT MVP

Design principle

One process, one laptop, no server you don't control. Every architectural choice below optimizes for "still works if the venue wifi dies five minutes before your slot."

Revised pipeline (unchanged in *logic*, changed in *implementation*)



Pre-hackathon / Hour 0-8 (offline, one-time):

SRTM DEM → slope, aspect, HAND, TWI → per_village_terrain.csv

IMDLIB historical rainfall (1 real extreme event, chosen district) → rainfall.csv

Hand-curated village table (population, households, health/road distance) → villages.csv

Hand-curated site table (land, water, power, health, school capacity) → sites.csv

~15-20 hand-logged landslide points (from Bhuvan map) + negative samples → labels.csv

Live app (Streamlit, single process, imports local modules directly — no network calls):

villages.csv + terrain.csv + rainfall.csv

→ hazard.py (deterministic threshold score; RF model as optional overlay)

→ vulnerability.py (weighted index from villages.csv)

→ risk.py (max-rule composite, banded)

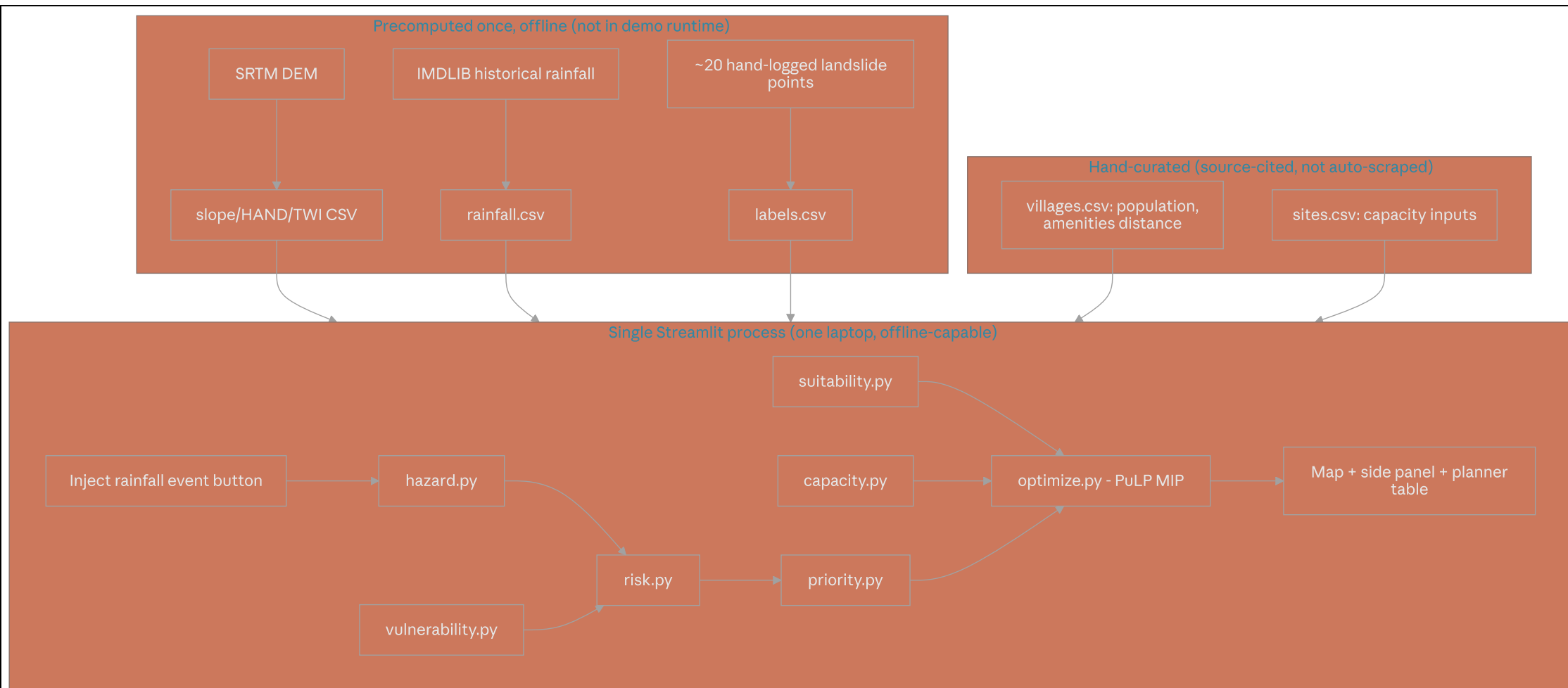
→ priority.py (weighted formula, tiered)

sites.csv → suitability.py + capacity.py (bottleneck-of-constraints)

priority + suitability + capacity → optimize.py (PuLP MIP, haversine distances)

"Inject rainfall event" button → re-runs hazard.py → risk.py → priority.py in-process, updates the map live

Revised architecture diagram



What survives from the original PRD, unchanged

- The priority-score formula and its full breakdown display (§9 FR-8/FR-9 of the original PRD)
- The carrying-capacity bottleneck logic and worked example
- The MIP allocation engine and its infeasibility-escalation behaviour
- The explainability-by-construction principle
- The live re-trigger demo beat

What changes

Layer	Was	Now
Frontend	React + MapLibre GL, separate build	Streamlit + PyDeck (or Folium) — single <code>streamlit run app.py</code> , no build step, no CORS, matches your team's already-proven Streamlit deployment experience
Backend	FastAPI service, called over HTTP	Plain Python modules , imported directly by the Streamlit app — no network hop, no service to keep alive
Database	PostgreSQL + PostGIS	CSV/GeoJSON files + pandas/GeoPandas in memory
Raster processing	Live GDAL/Rasterio in the app	Done once, offline, before the hackathon's live-coding clock starts ; only derived CSVs ship into the app
Routing	Self-hosted OSRM	Haversine straight-line distance , labelled honestly
Auth	RBAC + login + audit DB table	A view-as dropdown + in-memory action list
Hazard ML	Trained on NRSC's full inventory	Deterministic threshold score (guaranteed) + optional small-N RF trained on ~20 hand-logged points (stretch, disclosed honestly)
Deployment	Cloud VM, Docker Compose	Runs on the presenter's laptop , offline-capable; cloud link only as a pre-tested backup

Revised, genuinely-achievable data plan

Demo district: pick one with real, documented, public data behind it. Two strong, real options from the research: **Rudraprayag, Uttarakhand** (independently documented as India's most landslide-affected district — a defensible, citable choice) for a landslide-focused demo, or **Wayanad, Kerala** (documented landslide *and* flood history, e.g. the 2018 floods and the Puthumala landslide) if you want one district to carry a multi-hazard story. Recommend Wayanad if your narrative leans on "multi-hazard," Rudraprayag if it leans on "landslide depth." Verify actual data availability for whichever you pick before committing — don't assume, spend 30 minutes checking on day 1.

DEM: SRTM 30m, OpenTopography — download once, process once, ship the CSV.

Rainfall: IMDLIB historical gridded data for one real, named extreme-rainfall date in that district — this is your "replay" event.

Population/amenities: hand-curated CSV, 20–30 villages, sourced from the district's published Census Handbook + OpenStreetMap + Google Maps — cite your sources in an appendix slide, it reads as rigor, not a shortcut.

Landslide labels: ~15–20 points hand-logged from Bhuvan's public web map for that district, plus generated negative samples — disclosed on stage as a small-N proxy dataset.

Sites: 3–5 candidate relocation sites, capacity inputs estimated from published planning norms (persons/hectare, LPCD water norms) where you can't get exact local infrastructure figures — labelled as planning estimates, exactly as the original PRD already required.

Hour-by-hour build plan (assumes a ~48h clock starting at kickoff)

Window	Focus	Owner
0–4h	Pick district, confirm data availability, start hand-curating villages.csv/sites.csv	Data/GIS lead (start immediately — data curation is the real bottleneck, not code)
4–10h	SRTM download + slope/HAND/TWI computation (offline notebook); scaffold Streamlit app skeleton with tabs; write hazard.py/vulnerability.py against dummy data	GIS lead + Frontend lead in parallel
10–18h	Wire real computed CSVs into hazard/vulnerability/priority modules; build capacity.py bottleneck logic; start optimize.py (PuLP)	Backend/ML lead
18–26h	Map view (PyDeck/Folium) wired to real data; side-panel score breakdown; "inject rainfall event" button wired end-to-end	Frontend lead + Backend lead
26–32h	Sleep block. A 48-hour hackathon with no scheduled rest produces a worse demo, not a more ambitious one.	Everyone
32–40h	Small-N RF stretch model (if core loop is fully working — do not start this if it isn't); polish UI text/labels; build and test the deliberate infeasible-assignment case	ML lead + whoever's free

Window	Focus	Owner
40–46h	Rehearse the demo script twice end-to-end, fix whatever breaks, prep the one-slide "here's what we deliberately didn't build and why" honesty slide	Whole team
46–48h	Buffer. Do not be writing code in this window.	Everyone

One slide you should actually make

Judges reward engineering judgment. A single slide titled **"What we deliberately did not build, and why"** — listing 3–4 of the cuts from Part 1 (e.g., "no live IMD API — replayed a real historical event instead," "no PostGIS — this doesn't need a database at this scale yet") — will read as more technically mature than pretending the full production architecture got built in two days. Pair it with the one you *did* build hard: the optimization engine and the live recompute. That contrast is the pitch.